# Device Upgrade Automation with Claude: Research and Content Plan

Network and Infrastructure Automation · 3 days (8 hours per day) · Intermediate · Compiled 30 October 2026

This document is the working plan for building the course from its outline. It records where the course came from, why the outline is shaped the way it is, what goes into every module (lessons, teaching points, demo, lab, pitfalls and knowledge checks), the lab environment to prepare, and the sources behind each fact. Numbers in square brackets point to the reference list at the end; every URL there was opened and checked against the fact it supports.


- 1. Where this course came from
- 2. How the outline was shaped
- 3. Audience and prerequisites
- 4. Lab environment
- 5. Module-by-module content plan
- 6. Capstone plan
- 7. Promotional video: research behind the script
- References


## 1. Where this course came from

**Client input.**

> Client use case 2, 'Device Upgrade Automation': "Devices upgraded along with space clear, firmware upload, post and pre checks along with scheduler option." Use case 10, 'Cisco ACI Upgrade Automation': "Managing upgrade of ACI chassis as per process suggested by Cisco." The client's aim is "to do automation using Claude for all these content".

Upgrades are routine maintenance that teams still run by hand at two in the morning. NIST frames patching and upgrading as preventive maintenance that should be operationalized, not improvised [7]. Load balancer and fabric upgrades are where that is hardest: each vendor has its own space requirements, boot model and HA order [1][3][4][5].

Most failed windows come from the same few causes: not enough disk (NetScaler needs 7 GB free in /var [4]), an unverified image [2], the wrong HA order [1][3], a missing backup, or a window that runs out mid-pair while sync and mirroring are degraded [2][3]. Each one can be checked by code before the window opens.

This course teaches teams to let Claude read the fleet, draft plans and explain diffs, while deterministic code performs every space clear, install and reboot behind a ServiceNow approval. That split follows OWASP's guidance on limiting agent permissions and requiring human approval for high-impact actions [37]. The client also asked for Cisco ACI upgrades done the way Cisco prescribes, which the course covers with Cisco's own validator and order [20][21].

## 2. How the outline was shaped

| Decision | Rationale |
| --- | --- |
| One vendor-neutral lifecycle across F5, NetScaler, Alteon and ACI | The client asked for space clear, firmware upload, pre and post checks and a scheduler; a single lifecycle lets the same runner and guardrails cover every vendor. |
| Include one Cisco ACI module in a multi-vendor course | Use case 10 asks for ACI upgrades 'as per process suggested by Cisco' and has its own dedicated course; here one module shows how ACI fits the same lifecycle, following Cisco's pre-upgrade checklist and validator [20][21] rather than inventing a process, and leaves depth to the dedicated ACI course. |
| Space clear is its own module with allowlists and dry run | Disk space is the client's first named step and a top failure cause; vendor guidance lists exactly what can be cleared [15][10]. |
| Golden snapshot and diff instead of ad hoc post-checks | A versioned snapshot makes the post-check objective and gives the change record clear evidence. |
| Scheduler reads approved windows from ServiceNow | The client asked for a scheduler option; tying it to change windows [28] stops runs outside approval. |
| Claude proposes, code executes, a human approves every destructive step | Reboots and deletions are high impact; OWASP recommends human approval and minimal permissions [37]. |

## 3. Audience and prerequisites

| Role | What they need | Starting point |
| --- | --- | --- |
| Load balancer and data center engineers | Safe, repeatable upgrades across vendors | Strong on one vendor's GUI and CLI; limited automation |
| NOC automation leads | A pattern for using AI assistance in production change | Comfortable with Python and REST |
| Change and release managers | Evidence, approvals and window control | Strong on ServiceNow change process |
| Enterprise IT and network operations teams | To run and support the scheduled pipeline | Familiar with maintenance windows and runbooks |

### Prerequisites from the outline

- Basic TCP/IP and load balancer administration (F5, NetScaler or Radware)
- Comfort with a Linux shell, REST APIs and JSON
- Basic Python

Gemnoor Pvt. Ltd. - For more info - https://gemnoor.com/

- Familiarity with an AI coding assistant is helpful, not required

### Checks before enrolment

- Have you upgraded at least one load balancer or HA pair, or watched it done?
- Can you use a Linux shell to check disk usage and copy files?
- Can you call a REST API with curl and read the JSON?
- Can you read and run a short Python script?
- Do you know your organization's change approval process in ServiceNow?

### Course outcomes

- Model a vendor-neutral device upgrade lifecycle: inventory, pre-checks, space clear, firmware upload, install, post-checks, rollback
- Set up Claude Code, the Claude Agent SDK and MCP servers that expose F5 iControl REST, NetScaler NITRO, Radware Alteon / Cyber Controller and Cisco APIC APIs with explicit scopes
- Build deterministic Python adapters that clear disk space, upload and verify firmware and install to the inactive boot location, HA-aware
- Diff pre and post snapshots and let Claude produce a structured verdict with rollback guidance
- Gate every consequential action behind ServiceNow change approval, schedule runs inside approved windows and keep a complete audit trail

## 4. Lab environment

Each team gets a mixed fleet: an F5 BIG-IP VE HA pair, a NetScaler VPX HA pair, an Alteon VA or recorded Alteon responses, access to the Cisco DevNet Always-On ACI Simulator plus recorded APIC responses, a ServiceNow developer instance and a GitHub repository with a self-hosted runner in the lab network. All hostnames are fictional.

### Components

GenNoorTech
For more info - https://gennoor.com

| Component | Version or edition | Purpose | How to obtain | Notes |
|---|---|---|---|---|
| F5 BIG-IP VE HA pair (lb-f5-01, lb-f5-02) | Source 16.1.x or 17.1.x, target 17.5.x [1] | F5 staging, install, HA ordering and rollback | Trial keys through MyF5 or F5 lab licenses through a reseller | Allocate disk so the image, two volumes and UCS archives fit; keep the source ISO for rollback tests. |
| NetScaler VPX HA pair (lb-ns-01, lb-ns-02) | NetScaler 14.1 builds (source and target) | NetScaler staging, install and HA ordering | VPX Express needs no license file [40] | Express has 20 Mbps and 250 SSL sessions, enough for labs [40]; size /var for the 7 GB requirement [4]. |
| Radware Alteon VA (lb-rw-01, lb-rw-02) | Alteon 33.x; SDK minimum 31.0.10.0 or 32.2.2.0 [38] | Alteon facts, image selection and rollback | Evaluation license through Radware sales or partner | Public trial terms could not be confirmed; fall back to recorded responses. |
| Cisco ACI | DevNet Always-On ACI Simulator and recorded APIC responses | Validator review, APIC API reads and plan building | Free DevNet sandbox, no reservation [23] | Shared instance: read calls only; write payloads are demonstrated with recorded responses. |
| Anaya LB | Client-supplied | Adapter stub | Client to provide API documentation or a test device | No public documentation found. |
| ServiceNow Personal Developer Instance | Current release | CMDB, change windows and approvals | Free through the ServiceNow Developer Program [41] | Reclaimed after ten days unused [41]; provision close to the course and export the flows. |
| Claude Code and Claude Agent SDK | Current release | Workbench, planning and explanations | Official installer [32]; Agent SDK from the package index with an API key | Use organization API keys. |
| Python, Pydantic, pytest, APScheduler | Python 3.11 or later; APScheduler 3.x [30] | Runner, models, tests and in-house scheduler | Standard package index | Pin versions. |
| GitHub repository with Actions | GitHub.com or Enterprise | Scheduling, CI and evidence pull requests | One repository per team | Self-hosted runner for device access. |

## Fixtures and sample data

| Fixture | Contents | Used in |
|---|---|---|
| Recorded device responses | F5 volume and image, NITRO hanode and install, Alteon SDK reads, APIC queries, including errors | Claude Code, Agent SDK and MCP for upgrade workflows; Inventory and pre-flight discovery |
| CMDB fixture | Lab load balancers with one deliberate role mismatch | Inventory and pre-flight discovery |
| Disk fill script | Creates old images, cores, crash files and traces | Disk space clearing and firmware staging |
| ACI validator output | Recorded script output with PASS, FAIL and MANUAL results | Cisco ACI and fleet-scale, multi-vendor runs |
| Broken upgrade scenario | A pool that fails after upgrade | Post-checks, verification and rollback |

### Lab safety rules

- Lab devices only, fictional hostnames
- Read-only tools by default; destructive steps only through the runner with an Approved change
- Credentials only in environment variables or the lab vault
- Shared DevNet sandbox used for read calls only
- Reset scripts restore every device each evening

## 5. Module-by-module content plan

### Day 1

#### Module 1: The device upgrade lifecycle and why upgrades fail

**Goal.** Learners can describe how each vendor really upgrades and map every step to one vendor-neutral lifecycle before writing any automation.

**Outline topics.**

- Vendor upgrade mechanics side by side: F5 boot locations and UCS archives, NetScaler /var/nsinstall and build files, Radware Alteon image banks, Cisco ACI firmware groups
- The classic failure modes: full /shared or /var, corrupt image, HA pair upgraded in the wrong order, missing backup, change window overrun
- A vendor-neutral lifecycle: inventory, pre-check, space clear, upload, install, post-check, rollback, report
- The teaching pattern for the whole course: Claude plans and explains, deterministic code executes, a human approves, everything is logged

Genmoor Tech TRAINING INNOVATE BUILD for more info - https://genmoor.com

**Learning objectives.**

- Learners can compare F5, NetScaler, Alteon and Cisco ACI upgrade mechanics
- Learners can name the classic failure modes and the step where each shows up
- Learners can map a vendor runbook onto the lifecycle: inventory, pre-check, space clear, upload, install, post-check, rollback, report
- Learners can explain the course pattern: Claude plans and explains, code executes, a human approves, everything is logged

**F5 BIG-IP: boot locations and UCS**

- Images go in /shared/images and install to an inactive boot location with tmsh install sys software image <iso> volume HD1.x [1]
- The source must be on 15.x or later to go to 17.5.1.4, so older systems need more than one hop [1]
- Save a UCS in /var/local/ucs and copy it off the device before upgrading [1][11]
- Volumes report active, version, status and default boot location, which is how you know where you are [9]
- Install takes roughly three to seven minutes; failures are written to /var/log/liveinstall.log [1]

**NetScaler: /var/nsinstall and build files**

- Builds are copied to /var/nsinstall/<release>/build_<build>, extracted and installed with ./installns [14]
- ./installns --pre_check runs the pre-upgrade validation before the real install [14]
- /var needs 7 GB free and /flash at least 250 MB [4]
- Upgrade one major release at a time, for example 12.1 to 13.1 to 14.1 [4]

**Radware Alteon: image banks and no config rollback**

- Back up the configuration and run the Upgrade Advisor Tool against the current config and target version [5]
- 31.x to 33.x upgrade directly; older versions need intermediate hops [5]
- Configuration rollback is not supported: rollback means factory config for next boot, select the old image, reboot, upload the saved config [5]
- Some older Radware device families also need an upgrade password tied to the base MAC address; confirm for your Alteon model [6]

**Cisco ACI: controllers first, then switch groups**

- APICs are upgraded before switches [20]
- Since 4.0(1) there is one switch update group type; upgrade one switch of each HA pair at a time [20]
- The pre-upgrade validator runs when an update group is configured; a standalone script adds checks [20][21]

- All switches must be Active under Fabric Membership before starting [20]

**Why upgrades fail and the lifecycle**

- Full /shared or /var: the image is uploaded but the install has no room [4][15]
- Corrupt or unverified image: always check the vendor checksum before install [2][8]
- HA pair in the wrong order: upgrade the standby or secondary first [1][3]
- Missing backup or a backup left only on the device [1]
- Window overrun: F5 notes mirroring does not work until all devices run the new version, so a half-done pair is exposed [2]
- NIST frames upgrades as routine preventive maintenance to be operationalized, not ad hoc work [7]

**Instructor demo.**

1. Show the F5 volume list on lb-f5-01 through GET /mgmt/tm/sys/software/volume and tmsh show sys software
2. Show df -h on a NetScaler and point at /var usage against the 7 GB requirement
3. Walk the vendor runbooks side by side on screen
4. Fill the lifecycle map live with learner input
5. Show a failed install entry in /var/log/liveinstall.log from a recorded fixture
6. Explain the operating pattern and who does what at each stage

**Hands-on lab.** Walk a real F5 and NetScaler upgrade runbook by hand in the lab, record every step and decision, and map each to the lifecycle stages.

1. Read the F5 and NetScaler runbooks provided
2. On lb-f5-02 (standby) list volumes and images and record the current version
3. Save a UCS and copy it to the lab file server
4. On lb-ns-02 (secondary) check /var and /flash free space
5. Run ./installns --pre_check on the staged build and record the output
6. Record each decision point (go or stop) and who would make it
7. Map every step to a lifecycle stage on the template
8. Mark every point where one of the classic failures would appear

**Lab success criteria.**

- Every runbook step is mapped to a lifecycle stage
- Each classic failure mode is placed on the map with its detection point
- Version, disk space and backup location are recorded for both devices
- Decision points name the approver role

**Deliverable.** Annotated upgrade lifecycle map.

**Common pitfalls.**

- Treating the vendor upgrade guide as optional reading
- Forgetting multi-hop paths when the source is old
- Leaving the only backup on the device being upgraded
- Assuming Alteon can roll back configuration like F5 or NetScaler

**Knowledge check.**

1. Where does an F5 install go? Answer: To an inactive boot location (volume), for example HD1.2, while the active volume keeps running.
2. How much free space does NetScaler need in /var for an upgrade? Answer: 7 GB: 1 GB for the bundle, 2 GB for extraction and 4 GB for the process.
3. What is the rollback path on Alteon? Answer: Set next-boot config to factory, choose the previous image, reboot and upload the saved configuration.
4. In ACI, what is upgraded first? Answer: The APIC cluster, before any switches.

**Slides and assets to prepare.**

- Vendor mechanics comparison table
- Lifecycle map template
- Failure mode cards
- Recorded liveinstall.log failure
- Lab estate topology diagram

**Sources for this module.** 1, 2, 3, 4, 5, 6, 7, 8, 9, 11, 14, 15, 20, 21

## Module 2: Claude Code, Agent SDK and MCP for upgrade workflows

**Goal.** Learners build a workbench where Claude sees the fleet only through typed read-only tools.

**Outline topics.**

- Installing Claude Code, project CLAUDE.md conventions and a Python Agent SDK scaffold for the upgrade runner
- Writing an MCP server that wraps F5 iControl REST (/mgmt/tm/sys/version, /mgmt/tm/sys/software), NetScaler NITRO (/nitro/v1/config) and Radware Alteon REST, or Cyber Controller for centrally managed Alteons, as read-only tools
- Tool use with strict JSON schemas: every tool returns typed, validated output, never free text
- Separating what the model may propose from what the code may do

**Learning objectives.**

- Learners can install Claude Code and write a CLAUDE.md for the upgrade runner
- Learners can build an MCP server wrapping F5, NetScaler, Alteon and APIC read calls
- Learners can make every tool return validated typed output
- Learners can separate what the model may propose from what code may do

**Claude Code and project conventions**

- Install with curl -fsSL https://claude.ai/install.sh | bash; use claude -p for scripted runs [32]
- Project instructions in ./CLAUDE.md or ./.claude/CLAUDE.md, shared through source control [42]
- Record hostnames, version matrix location, naming rules and red lines (no install, reboot or delete tools)
- Keep the file short and specific so it is followed [42]

**Agent SDK scaffold**

- Define tools with @tool and register them with create_sdk_mcp_server [33]
- Tool names become mcp__upgrader__get_version and must be listed in allowed_tools [33]
- Return structuredContent and isError: true on failure [33]
- Pre-approve read tools only; anything else must prompt or be absent

**Read-only device tools**

- F5: GET /mgmt/tm/sys/software/volume and /mgmt/tm/sys/software/image for versions, boot locations and images [9][8]
- NetScaler: GET /nitro/v1/config/hanode for HA state and sync [18]
- Alteon: software and facts through the Radware SDK management functions [38]
- APIC: log in with POST /api/aaaLogin.json, then read-only class queries [24]
- Cyber Controller for centrally managed Alteons is an option; confirm its API with Radware before building on it

**Strict schemas and the propose-versus-do line**

- Every tool output is a Pydantic model validated before it is returned [31]
- Mark tools with readOnlyHint, but remember annotations are only hints and enforcement stays in code [34]
- The model may propose plans and explain; only the runner may call write endpoints
- Plans the model proposes are JSON validated against a schema [35]

**Instructor demo.**

1. Create the repo and CLAUDE.md live
2. Write get_version for F5 against a recorded volume response
3. Write get_ha_state for NetScaler and show the typed output
4. Register the server and ask Claude to describe the fleet
5. Ask Claude to reboot a device and show there is no tool to do it
6. Show a tool returning isError when a device is unreachable

**Hands-on lab.** Build the read-only MCP server, connect it to Claude Code and ask Claude to describe the fleet using only tool results.

1. Clone the starter repo and set up a virtual environment

2. Write CLAUDE.md with conventions and red lines
3. Implement get_version, get_images and get_ha_state for F5
4. Implement get_version and get_ha_state for NetScaler
5. Implement get_version for Alteon against recorded responses
6. Implement get_firmware_status for APIC against the DevNet sandbox or recorded responses
7. Validate every output with Pydantic and add tests
8. Ask Claude to describe the fleet and check each statement against tool output

**Lab success criteria.**

- All tools return validated typed output
- Claude's fleet description matches tool output with no invented facts
- No write tool exists in the server
- Unreachable devices produce isError results, not guesses

**Deliverable.** Working MCP server and agent scaffold.

**Common pitfalls.**

- A generic shell tool that bypasses the read-only design
- Free-text tool output
- Trusting tool annotations as enforcement
- Device credentials in CLAUDE.md

**Knowledge check.**

1. What is the full tool name Claude sees for get_version on server upgrader? Answer: mcp__upgrader__get_version.
2. Are MCP tool annotations enforcement? Answer: No; they are hints, and clients should not rely on annotations from untrusted servers.
3. Which F5 endpoint lists boot locations? Answer: /mgmt/tm/sys/software/volume.

**Slides and assets to prepare.**

- Starter repo and CLAUDE.md template
- Tool catalogue with schemas
- Recorded device responses for all four vendors
- Workbench architecture diagram

**Sources for this module.** 8, 9, 18, 24, 31, 32, 33, 34, 35, 38, 42

## Module 3: Inventory and pre-flight discovery

**Goal.** Learners reconcile CMDB records with live device facts into one typed inventory and readiness report.

**Outline topics.**

GennoorTech PLAN. INNOVATE. BUILD. For more info - https://gennoor.com

- Pulling the device inventory and owners from the ServiceNow CMDB with the Table API (cmdb_ci_lb_* tables) and reconciling it with live device facts
- Collecting version, licence, platform, HA role, sync state and disk usage per vendor into one Pydantic inventory model
- Target version matrix and upgrade path rules (for example multi-hop F5 paths) as data, not prompt text
- Claude summarises readiness per device and flags inconsistencies between CMDB and reality

**Learning objectives.**

- Learners can query ServiceNow CMDB load balancer tables through the Table API
- Learners can collect version, HA role, sync state and disk usage per vendor
- Learners can express the target version matrix and upgrade paths as data
- Learners can have Claude summarize readiness and flag CMDB mismatches

**CMDB as the starting list**

- Load balancers live in cmdb_ci_lb and classes such as cmdb_ci_lb_bigip and cmdb_ci_lb_netscaler [26]
- Query with GET /api/now/table/cmdb_ci_lb_bigip?sysparm_query=...&sysparm_fields=name,ip_address,... [25]
- Page large results with sysparm_limit and sysparm_offset [25]
- Record owner and support group for each device for later approvals

**Live facts per vendor**

- F5: active volume version and build, images present, HA state [9][8]
- NetScaler: master state Primary or Secondary and sync status from hanode [18]
- Alteon: software version through SDK management info [38]
- Disk usage: /shared on F5 and /var and /flash on NetScaler against vendor minimums [4]

**The inventory model**

- One Pydantic model: hostname, vendor, platform, version, ha_role, sync_state, disk, license_ok, owner, cmdb_sys_id [31]
- Validation errors are reported per device, not hidden [31]
- A mismatch field records where CMDB and the device disagree
- Each record keeps the raw source responses by reference so reviewers can check values

**Version matrix and paths as data**

- targets.yaml: vendor, platform, target version, allowed source versions
- Path rules: F5 17.5.1.4 needs 15.x or later at source [1]; NetScaler one major release at a time [4]; Alteon older than 31.x needs hops [5]
- The planner reads paths from data, never from the prompt

- Each device gets READY, NOT READY or NEEDS HOP with reasons

**Claude summarizes readiness**

- Claude reads the inventory and writes a per-device readiness summary
- Every NOT READY device gets a plain-English reason tied to a field
- CMDB mismatches are listed for the CMDB owner, not fixed by the agent
- Summaries cite inventory fields by name so a reviewer can check every claim

**Instructor demo.**

1. Query CMDB for lab load balancers and show the JSON
2. Collect live facts from the lab fleet
3. Show a CMDB mismatch on lb-ns-03 role
4. Load targets.yaml and compute readiness
5. Ask Claude to explain each not-ready device

**Hands-on lab.** Generate the readiness report for a mixed lab fleet and have Claude explain every device marked not ready.

1. Write the CMDB query for the lab tables
2. Implement live-fact collectors for each vendor
3. Define the inventory Pydantic model
4. Write targets.yaml with lab target versions
5. Implement readiness rules from the matrix
6. Produce readiness.json and readiness.md
7. Ask Claude for explanations and review them against data
8. Write tests with recorded CMDB and device responses

**Lab success criteria.**

- Every lab device appears with readiness and reasons
- CMDB mismatches are listed
- Path rules come from data files
- Tests pass offline

**Deliverable.** Fleet readiness report.

**Common pitfalls.**

- Trusting CMDB HA role without checking the device
- Encoding upgrade paths in prompts
- Ignoring /flash on NetScaler
- Letting the agent update CMDB

**Knowledge check.**

1. Which table holds F5 BIG-IP CIs? Answer: cmdb_ci_lb_bigip.

Gennoor Tech TRAIN. INNOVATE. BUILD. For more info - https://gennoor.com

2. Why keep upgrade paths in data? Answer: So they are reviewed, versioned and enforced in code rather than depending on the model.
3. What happens to a CMDB mismatch? Answer: It is reported to the CMDB owner and blocks readiness until resolved; the agent does not fix it.

**Slides and assets to prepare.**

- CMDB field map
- targets.yaml example
- Readiness report sample
- Mismatch report template

**Sources for this module.** 1, 4, 5, 8, 9, 18, 25, 26, 31, 38

## Module 4: Pre-checks and the golden snapshot

**Goal.** Learners capture a versioned snapshot and backups before any change and stop the run when anything is red.

**Outline topics.**

- Virtual server, pool member and monitor status, HA failover and config-sync state, certificate expiry, licence validity
- Configuration backups before any change: F5 UCS via tmsh save sys ucs, NetScaler system backup via NITRO, Alteon configuration export
- A versioned JSON snapshot schema that the post-check module will diff against
- Hard stop rules: an upgrade does not proceed when any pre-check is red or a backup is missing

**Learning objectives.**

- Learners can collect virtual server, pool, HA, certificate and licence state
- Learners can take and verify backups on each vendor
- Learners can define a versioned JSON snapshot schema
- Learners can enforce hard stop rules

**What goes in the snapshot**

- Virtual servers and their status, pools and member status, monitors
- HA role, failover state and config-sync status [18][39]
- Certificate expiry for bound certificates
- Version, active volume and licence validity [9]

**Backups before change**

- F5: tmsh save sys ucs <name> writes to /var/local/ucs [11]; download it with GET /mgmt/shared/file-transfer/ucs-downloads/<name> [13]

Gemnoor Tech TRAINING INNOVATE BUILD For more info - https://gemnoor.com

- NetScaler: POST /nitro/v1/config/systembackup?action=create with level full [17]
- Alteon: export and save the configuration before upgrade [5]
- Copy every backup off the device and record a hash of the file

**Snapshot schema**

- snapshot_id, device, captured_at, schema_version, sections{}
- Stable sort order so diffs are meaningful
- Stored in the repo or artifact store, not only on disk
- Secrets and session tokens are never written into a snapshot

**Hard stop rules**

- Any red pre-check stops the run
- No backup or an unreadable backup stops the run [1]
- Sync not in sync on an HA pair stops the run
- Claude may explain the stop but cannot override it

**Instructor demo.**

1. Capture a snapshot of lb-f5-01 and lb-f5-02
2. Save and download a UCS
3. Create a NetScaler backup through NITRO
4. Corrupt the backup file and rerun to show the stop
5. Ask Claude to explain the stop reason

**Hands-on lab.** Implement the pre-check and snapshot module with pytest coverage, then break a backup on purpose and watch the run stop.

1. Define the snapshot schema
2. Implement collectors for each section
3. Implement UCS save and download for F5
4. Implement systembackup for NetScaler
5. Implement config export for Alteon
6. Implement the stop rules
7. Write pytest cases for pass and stop paths
8. Break the backup and confirm the run stops

**Lab success criteria.**

- Snapshot saved with schema version and ID
- Backups exist off-device with hashes
- A missing or corrupt backup stops the run with a logged reason
- Tests cover each stop rule

**Deliverable.** Pre-check gate and golden snapshot.

Gennoor Tech TRAIN. INNOVATE. BUILD. For more info - https://gennoor.com

**Common pitfalls.**

- Backups left only on the device
- Snapshots with unstable ordering
- Treating warnings as passes without a rule
- Skipping the standby

**Knowledge check.**

1. Where does tmsh save sys ucs write? Answer: /var/local/ucs.
2. What does the run do if a backup is missing? Answer: It stops; no backup, no upgrade.
3. Why version the snapshot schema? Answer: So post-check diffs compare like with like after the schema changes.

**Slides and assets to prepare.**

- Snapshot schema
- Stop rule table
- Backup procedures per vendor
- Corrupt backup fixture

**Sources for this module.** 1, 5, 9, 11, 13, 17, 18, 39

## Day 2

### Module 5: Disk space clearing and firmware staging

**Goal.** Learners free space safely from allowlisted paths and stage verified firmware.

**Outline topics.**

- Safe space reclamation per vendor: old F5 software images and UCS archives, NetScaler /var/nsinstall, /var/core and /var/crash, stale Alteon images
- Allowlists of deletable paths and a dry-run mode that reports what would be removed and how much space it frees
- Firmware upload with iControl REST software-image uploads and NITRO systemfile, with SHA256 verification against the vendor checksum
- Claude drafts the space-clear plan from disk facts; the adapter executes only allowlisted deletions after approval

**Learning objectives.**

- Learners can identify safe deletions per vendor
- Learners can produce a dry-run space plan with sizes
- Learners can upload firmware and verify its checksum
- Learners can execute only approved, allowlisted deletions

Gennoor Tech
INNOVATE. BUILD. TRANSFORM.
For more info - https://gennoor.com

**Safe reclamation per vendor**

- F5: delete old images with tmsh delete sys software image <name> or DELETE on /mgmt/tm/sys/software/image [10][8]
- F5: remove old UCS archives with delete ucs after confirming off-device copies [11]
- NetScaler: clear /var/core and /var/crash unless needed, /var/nsinstall except current firmware, /var/tmp/support bundles [15]
- NetScaler: /var/nstrace is the most common cause of a full disk [15]
- Alteon: remove stale images only per Radware guidance for your model

**Allowlists and dry run**

- Allowlist per vendor of directories and file patterns
- Never delete the running image or the image in the active or default boot location [9]
- Dry run lists each file, size and total freed
- Claude drafts the plan from disk facts; code checks every item against the allowlist

**Firmware upload**

- F5: POST /mgmt/cm/autodeploy/software-image-uploads/<iso> lands in /shared/images with Content-Range chunking [13]
- NetScaler: copy the build to /var/nsinstall/<release>/build_<build> [14]
- Check free space against the vendor minimum before upload [4]
- Resume interrupted uploads by chunk rather than restarting from zero [13]

**Checksum verification**

- Compare a locally computed hash with the vendor-published checksum; F5's upgrade guide uses md5 [2]
- F5 also reports checksum and verified for each image [8]
- Store the hash in the run state
- Stop on mismatch and delete the bad upload

**Instructor demo.**

1. Fill /var on the lab NetScaler with fixture files
2. Run the dry-run plan and show totals
3. Show an item refused because it is outside the allowlist
4. Approve and execute
5. Upload the F5 image and verify the checksum

**Hands-on lab.** Fill a lab device disk, run the dry-run plan, approve it and stage the firmware image with checksum verification.

1. Fill the lab disk with the fixture script
2. Collect disk facts

Genroot Tech TRAINING INNOVATE BUILD For more info - https://genroot.com

3. Ask Claude to draft the space plan
4. Validate the plan against the allowlist
5. Run dry run and review
6. Obtain approval on the lab change
7. Execute deletions
8. Upload firmware and verify checksum

**Lab success criteria.**

- Free space meets the vendor minimum
- Only allowlisted files were deleted
- Firmware checksum matches the vendor value
- All actions logged with approval reference

**Deliverable.** Space-clear and staging adapter.

**Common pitfalls.**

- Deleting the image on the rollback volume
- Deleting core files someone is investigating
- Uploading before checking space
- Comparing different hash algorithms

**Knowledge check.**

1. What is the most common cause of a full NetScaler disk? Answer: Trace files in /var/nstrace left running.
2. Which F5 endpoint accepts image uploads? Answer: /mgmt/cm/autodeploy/software-image-uploads/, writing to /shared/images.
3. What does the dry run show? Answer: Every file to be deleted, its size and the total space freed.

**Slides and assets to prepare.**

- Allowlist per vendor
- Disk fill fixture script
- Dry-run report sample
- Checksum verification flow

**Sources for this module.** 2, 4, 8, 9, 10, 11, 13, 14, 15

## Module 6: Upgrade execution adapters, HA-aware

**Goal.** Learners build a runner that installs to the inactive location, upgrades HA pairs in the right order and can resume after failure.

**Outline topics.**

Gennoor Tech
INNOVATE. BUILD.
for more info: https://gennoor.com

- F5: install to the inactive boot volume via iControl REST or tmsh install sys software image, then reboot into it; NetScaler: NITRO install and reboot; Alteon: image bank update and reload
- HA ordering: upgrade standby, verify, fail over, upgrade the former active, confirm sync
- Idempotent, resumable steps with state recorded after every action so a failed run can continue or roll back
- Claude proposes the ordered plan and the exact API calls as structured JSON; the runner executes the approved plan only

**Learning objectives.**

- Learners can install and reboot on each vendor through API or CLI
- Learners can order HA upgrades correctly
- Learners can make steps idempotent and resumable
- Learners can run only an approved structured plan

**Vendor install steps**

- F5: tmsh install sys software image <iso> volume HD1.x, or POST to /mgmt/tm/sys/software/image; watch status on the volume [1][9]
- F5: reboot into the new volume with the reboot option or by setting the boot location [10]
- NetScaler: POST /nitro/v1/config/install with url and Async to get a job id, then reboot [16][19]
- Alteon: set the image for next boot and reload, following the release notes [5]

**HA ordering**

- Upgrade standby or secondary first and verify [1][3]
- F5: run sys failover standby on the active to move traffic [12]
- NetScaler: force failover so the upgraded node becomes primary [3]
- Upgrade the former active, then confirm sync [39]
- Expect sync and mirroring limits while versions differ [2][3]

**Idempotent and resumable**

- State file per run: step, device, status, timestamps, request ids
- Each step checks current state first: if the target volume already runs the target version, skip
- Resume from the last completed step after a crash
- Rollback path recorded before each destructive step

**Plan in, actions out**

- Claude proposes the ordered plan and exact calls as JSON validated against a schema [35]
- The runner executes only the approved plan hash
- Any deviation stops the run
- Human approval required before install and before each reboot

**Instructor demo.**

Genmoor Tech
For more info - https://genmoor.com

1. Show the plan JSON for an F5 pair
2. Run install on the standby and watch status
3. Reboot and verify version
4. Fail over and upgrade the former active
5. Kill the runner halfway and resume

**Hands-on lab.** Upgrade an HA pair of lab load balancers end to end through the runner, with a mocked failure halfway to test resumption.

1. Write the plan schema
2. Implement F5 install, reboot and failover steps
3. Implement NetScaler install and reboot steps
4. Implement the state file
5. Add idempotency checks
6. Run on the F5 lab pair
7. Inject a failure after the standby reboot
8. Resume and complete

**Lab success criteria.**

- Standby upgraded and verified before failover
- Both units on target version and in sync
- Resume continues from the right step
- No step outside the plan ran

**Deliverable.** HA-aware upgrade runner.

**Common pitfalls.**

- Upgrading the active first
- Not waiting for install status complete before reboot
- Losing state on crash
- Assuming sync is normal while versions differ

**Knowledge check.**

1. Which unit is upgraded first? Answer: The standby or secondary.
2. How does NetScaler async install report progress? Answer: It returns an install id tracked with show ns job.
3. Why record state after every step? Answer: So a failed run can resume or roll back from a known point.

**Slides and assets to prepare.**

- HA upgrade sequence diagram
- Plan schema
- State file example

Gennoor Tech TRAIN · INNOVATE · BUILD · For more info - https://gennoor.com

- Failure injection script

**Sources for this module.** 1, 2, 3, 5, 9, 10, 12, 16, 19, 35, 39

## Module 7: Post-checks, verification and rollback

**Goal.** Learners compare the post-upgrade state with the golden snapshot, get a structured verdict and execute rollback when needed.

**Outline topics.**

- Capturing the post-upgrade snapshot and diffing it against the golden snapshot: version, virtual server status, pool members, HA sync, certificates
- Claude reads the diff and returns a structured verdict (pass, warn, fail) with a plain-English explanation for the change ticket
- Rollback paths: boot the previous F5 volume, NetScaler previous build, Alteon previous image bank, restore from backup
- Evidence pack generation for the change record

**Learning objectives.**

- Learners can diff post and golden snapshots
- Learners can get a structured pass, warn or fail verdict
- Learners can execute vendor rollback paths
- Learners can generate an evidence pack

**The diff**

- Same schema as the golden snapshot so sections compare directly
- Version must change; virtual servers, pools and members must not degrade
- HA sync may show changes pending until the peer is upgraded; treat as expected only in that window [2][3]
- Certificate expiry should be unchanged

**Structured verdict**

- Claude reads the diff and returns verdict, reasons and rollback recommendation as JSON matching a schema [35]
- Code applies hard rules first: any fail rule overrides the model
- The verdict text goes into the change record
- A warn verdict needs a named human to accept it before the run continues

**Rollback paths**

- F5: boot back into the previous volume, which is still installed [9][10]
- F5: restore configuration with load ucs if needed [11]
- NetScaler: reinstall the previous build retained in /var/nsinstall [14][15]

GenNoorTech INNOVATE BUILD
For more info https://gennoor.com

- Alteon: factory config next boot, previous image, reboot, upload saved config [5]

**Evidence pack**

- Pre and post snapshots, diff, verdict, logs and approvals
- Attach to the change record with a PATCH work note [25]
- Store hashes of each file in the pack
- Name the pack with the change number and run id so audits can find it

**Instructor demo.**

1. Diff a clean upgrade
2. Diff a broken one with a pool down
3. Show the verdict JSON
4. Roll back the broken device
5. Generate the evidence pack

**Hands-on lab.** Run the post-check diff on a successful and an intentionally broken upgrade, then execute the rollback path for the broken one.

1. Implement the diff
2. Define the verdict schema
3. Implement hard rules
4. Ask Claude for the verdict on both runs
5. Execute rollback for the broken device
6. Re-run post-checks after rollback
7. Build the evidence pack
8. Attach it to the change

**Lab success criteria.**

- Clean run verdict pass
- Broken run verdict fail with rollback recommended
- Rollback restores the golden snapshot state
- Evidence pack complete

**Deliverable.** Verification diff and rollback module.

**Common pitfalls.**

- Letting the model override a hard fail
- Deleting the previous volume or build before verification
- Diffing snapshots with different schemas
- Forgetting Alteon has no config rollback

**Knowledge check.**

Gennoor Tech TRAIN · INNOVATE · BUILD For more info - https://gennoor.com

1. What is the F5 rollback? Answer: Boot the previous volume, which remains installed.
2. Who has the final say on fail? Answer: Code hard rules; the model explains.
3. What goes in the evidence pack? Answer: Snapshots, diff, verdict, logs and approvals with file hashes.

**Slides and assets to prepare.**

- Diff output sample
- Verdict schema
- Rollback runbook per vendor
- Evidence pack template

**Sources for this module.** 2, 3, 5, 9, 10, 11, 14, 15, 25, 35

## Module 8: Guardrails, approvals and audit for upgrades

**Goal.** Learners gate every space clear, install and reboot behind ServiceNow approval and prove the gate cannot be talked around.

**Outline topics.**

- Allowlisted tools and devices per run, read-only by default, destructive actions behind explicit approval
- ServiceNow change requests created through the Table API and approved through Flow Designer before space clear, install and reboot
- An append-only audit log of every prompt, plan, approval and API call, with secrets kept out of prompts and logs
- Testing the guardrails with pytest: prompt the agent to skip a check and prove the runner refuses

**Learning objectives.**

- Learners can allowlist tools and devices per run
- Learners can create change records and wait for approval
- Learners can keep an append-only audit log without secrets
- Learners can test that the runner refuses prompts to skip checks

**Allowlists and read-only default**

- Each run has an allowlist of devices and actions
- Read-only by default; destructive actions need explicit approval
- OWASP recommends minimal permissions and human approval for high-impact actions [37]
- Device service accounts for the runner have only the rights the plan needs [37]

**ServiceNow approvals**

- Create the change through the Table API or Change Management API [25][28]

- Use Ask for Approval before space clear, install and reboot; only Approved proceeds [27]
- Avoid parallel Ask for Approval actions on the same record [27]
- Read approval state at run time, not at plan time

**Audit log**

- Append-only JSON lines for prompts, plans, approvals and API calls
- Redact secrets and tokens
- Each entry has run id, change number and step
- Ship entries off the runner host as they are written so they cannot be edited later

**Testing the guardrails**

- pytest cases that prompt the agent to skip a pre-check and assert the runner refuses
- PreToolUse hooks deny shell commands that call device write APIs directly [36]
- Tests for every approval state
- Run the guardrail tests in CI on every pull request

**Instructor demo.**

1. Attempt an unapproved reboot
2. Approve and retry
3. Prompt the agent to skip a check and show refusal
4. Open the audit log

**Hands-on lab.** Wire the approval gate to a lab ServiceNow instance and attempt an unapproved reboot; inspect the audit trail afterwards.

1. Implement the allowlist
2. Implement change creation
3. Build the approval flow
4. Implement the gate before destructive steps
5. Implement the audit log
6. Write guardrail tests
7. Attempt the unapproved reboot
8. Inspect the audit trail

**Lab success criteria.**

- Unapproved reboot refused with logged reason
- Approved reboot proceeds
- Skip-check prompt refused
- Audit log complete and secret-free

**Deliverable.** Approval gate and audit log.

**Common pitfalls.**

Gennoor Tech TRAINING INNOVATE GUIDE For more info - https://gennoor.com

- Treating Requested as approved
- Approval checked only once
- Secrets in logs
- Hooks as the only control

**Knowledge check.**

1. Which approval states allow a reboot? Answer: Only Approved.
2. What does a PreToolUse hook do here? Answer: Denies shell commands that would call device write APIs directly.
3. Why read approval at run time? Answer: Approvals can be withdrawn or windows can change after planning.

**Slides and assets to prepare.**

- Guardrail decision table
- Approval flow export
- Audit log schema
- Guardrail test suite

**Sources for this module.** 25, 27, 28, 36, 37

## Day 3

### Module 9: Scheduler and change windows

**Goal.** Learners schedule upgrades only inside approved windows and handle overruns safely.

**Outline topics.**

- Reading approved change windows from ServiceNow and refusing to start outside them
- Scheduling runs with GitHub Actions schedule and workflow_dispatch, or an in-house APScheduler service, with a persistent job state store
- Pre-window reminders, in-window progress and post-window summaries to the operations channel
- Handling overruns: pause at the next safe step, request a window extension, never leave a device mid-upgrade

**Learning objectives.**

- Learners can read change windows from ServiceNow
- Learners can schedule runs with GitHub Actions or APScheduler
- Learners can send reminders, progress and summaries
- Learners can pause safely on overrun

**Windows from ServiceNow**

Gennoor Tech TRAIN INNOVATE BUILD For more info - https://gennoor.com

- Read start_date and end_date from the change [28]
- Refuse to start outside the window
- Check conflicts with the Change Management API [28]
- Store the window in run state so a restarted runner still knows the deadline

**Scheduling options**

- GitHub Actions schedule runs in UTC (or a set timezone) on the default branch, at most every 5 minutes, and can be delayed at the top of the hour [29]
- workflow_dispatch with inputs for manual starts [29]
- APScheduler with a persistent job store such as SQLAlchemyJobStore on PostgreSQL [30]
- Set misfire_grace_time so a missed start inside the window still runs, but not after it closes [30]

**Notifications**

- Pre-window reminder with plan and approver
- In-window progress per device
- Post-window summary with verdicts
- Messages carry the change number and a link to the evidence pull request

**Overruns**

- Estimate time per step from history
- Before each step, check remaining window time
- Pause at the next safe step and request an extension
- Never leave a device mid-install or mid-reboot

**Instructor demo.**

1. Show a change window and the schedule
2. Start a run automatically
3. Shorten the window and show the pause
4. Show notifications

**Hands-on lab.** Schedule a batch upgrade in the lab, let it start automatically inside the window and watch it pause when the window is about to close.

1. Read the lab change window
2. Configure the scheduled workflow or APScheduler job
3. Add the window check
4. Add step time estimates
5. Add notifications
6. Start the batch
7. Shorten the window
8. Observe the safe pause

Genroor Tech
INNOVATE · BUILD
For more info - https://genroor.com

**Lab success criteria.**

- Run starts only inside the window
- Pause happens at a safe step
- Notifications sent
- State allows resume next window

**Deliverable.** Scheduled upgrade pipeline.

**Common pitfalls.**

- Time zone errors
- Scheduling at minute zero
- No persistent job store
- Pausing mid-reboot

**Knowledge check.**

1. Which branch does a scheduled workflow run on? Answer: The latest commit on the default branch.
2. What does misfire_grace_time control? Answer: Whether a missed run time still triggers when the scheduler comes back.
3. What is a safe step? Answer: A point where the device is fully on one version and in a known state.

**Slides and assets to prepare.**

- Window and schedule diagram
- Workflow file
- APScheduler config
- Notification templates

**Sources for this module.** 28, 29, 30

## Module 10: Cisco ACI and fleet-scale, multi-vendor runs

**Goal.** Learners plan ACI upgrades the way Cisco prescribes and run large fleets in safe batches.

**Outline topics.**

- Cisco ACI upgrades as Cisco prescribes: the ACI pre-upgrade validator, APIC firmware repository and firmware groups via the APIC REST API, APIC cluster first then switches in groups
- Batch strategies for large fleets: blast radius limits, canary devices and stop-on-first-failure
- Extending the adapter pattern to other load balancers (Anaya LB) with the same lifecycle contract
- Reporting in GitHub pull requests: plan, evidence pack and audit summary reviewed before the next batch

Genmoor Tech TRAIN · INNOVATE · BUILD · for more info - https://genmoor.com

**Learning objectives.**

- Learners can run the ACI pre-upgrade validator and read its results
- Learners can plan APIC-first upgrades and switch update groups
- Learners can design batches with canaries and blast radius limits
- Learners can extend the adapter contract to other load balancers (Anaya LB)

**ACI as Cisco prescribes**

- Export configuration with AES encryption first [20]
- Run aci-preupgrade-validation-script.py on the APIC from /data/techsupport and fix every FAIL and MANUAL CHECK [21]
- Checks include target image and MD5, APIC cluster Fully Fit, switch membership and disk usage [22]
- Upgrade the APIC cluster first, then switches [20]
- Put the two switches of each pair in different update groups and upgrade one group at a time [20]

**APIC API in the lab**

- Authenticate with POST /api/aaaLogin.json [24]
- Use the DevNet Always-On ACI Simulator for read calls [23]
- Capture write payloads from the APIC API Inspector in your own lab before automating them
- Treat the built-in validator as a gate, not just a report [20]

**Batch strategies**

- Canary first, then batches
- Blast radius: maximum HA pairs in flight
- Stop on first failure
- Re-plan between batches

**Other load balancers (Anaya LB) and reporting**

- Same lifecycle contract: precheck, clear, stage, install, postcheck, rollback
- Anaya LB adapter stub until the client supplies API details
- Pull request per batch with plan, evidence and audit summary
- claude -p writes the batch summary in CI [32]

**Instructor demo.**

1. Run the validator script output fixture
2. Show APIC login and a firmware status read on the sandbox
3. Build the update group plan
4. Run a canary-first batch

GenNoorTech TRAIN · INNOVATE · BUILD
for more info - https://gennoor.com

**Hands-on lab.** Run the ACI pre-upgrade validator and build an APIC firmware group plan, then execute a canary-first batch across the mixed lab fleet.

1. Review validator output
2. Classify results
3. Write the ACI plan
4. Check pair separation
5. Define batch policy
6. Run the canary
7. Run the batch
8. Open the pull request

**Lab success criteria.**

- Validator issues triaged
- APIC-first plan with separated pairs
- Canary succeeds before batch
- Batch stops on injected failure

**Deliverable.** Fleet batch runner and ACI plan.

**Common pitfalls.**

- Both pair members in one group
- Ignoring MANUAL CHECK results
- Using an old copy of the script
- No canary

**Knowledge check.**

1. What is upgraded first in ACI? Answer: The APIC cluster.
2. Where is the validation script run? Answer: On the APIC, typically from /data/techsupport.
3. Why separate pair members? Answer: So both members never reboot together.

**Slides and assets to prepare.**

- ACI upgrade sequence
- Validator output fixture
- Batch policy template
- PR template

**Sources for this module.** 20, 21, 22, 23, 24, 32

## Module 11: Capstone: scheduled, approved, HA-aware upgrade end to end

**Goal.** Teams run the full pipeline from device list to evidence pack and defend it.

Gennoor Tech
TRAIN · INNOVATE · BUILD
For more info - https://gennoor.com

**Outline topics.**

- Inventory in, verified upgrade out: pre-checks, space clear, upload, install, post-checks and rollback readiness on a mixed fleet
- Wire the scheduler, approval gate, HA-aware runner and post-check diff into one pipeline driven by a device list and target version matrix
- Evidence pack and audit summary delivered as a GitHub pull request for review
- Acceptance criteria review, limitations statement and live demonstration to the group

**Learning objectives.**

- Learners can run the pipeline end to end on a mixed fleet
- Learners can show approvals for every destructive step
- Learners can present evidence and limitations

**Brief**

- Device list and target version matrix in
- Pre-checks, space clear, upload, install, post-checks, rollback readiness
- Scheduler, approval gate, HA-aware runner and diff wired together
- Evidence pack and audit summary as a GitHub pull request

**Acceptance review**

- Trace one device from inventory to evidence
- Show a blocked step and a rollback
- Zero reboots without approval
- Limitations: Anaya LB, Alteon without a VA, ACI writes

**Demo and handover**

- Ten-minute demo
- Operator runbook
- Next batch plan
- Open questions for the client

**Instructor demo.**

1. Show the acceptance checklist
2. Walk a sample evidence pack
3. Explain scoring
4. Show a limitations statement

**Hands-on lab.** Take a mixed lab fleet from device list to evidence pack through the full pipeline, then demo the run, the approvals and the audit trail to the group.

1. Load the device list and matrix
2. Run inventory and readiness

Gennoor Tech TRAIN · INNOVATE · BUILD
For more info - https://gennoor.com

3. Run pre-checks and snapshot
4. Get approvals
5. Run space clear and staging
6. Run HA-aware upgrades
7. Run post-checks and any rollback
8. Open the evidence pull request and demo

**Lab success criteria.**

- All in-scope devices upgraded or rolled back with evidence
- Every destructive step has an approval
- Audit log complete
- Runbook and limitations delivered

**Deliverable.** End-to-end upgrade automation and runbook.

**Common pitfalls.**

- Skipping blocked and rollback cases
- Polishing code instead of evidence
- No limitations statement

**Knowledge check.**

1. What must every reboot have? Answer: An Approved change and an audit entry.
2. What goes in the pull request? Answer: Plan, evidence pack and audit summary.
3. What is the Anaya LB status? Answer: Adapter stub awaiting client API details.

**Slides and assets to prepare.**

- Acceptance checklist
- Scoring rubric
- Runbook template

**Sources for this module.** 20, 25, 27, 29

## 6. Capstone plan

A repeatable upgrade workflow that takes a device list and target versions, runs pre-checks, clears space, stages verified firmware, upgrades HA pairs in the right order inside an approved change window, verifies the result against a golden snapshot and ships an evidence pack, with no reboot ever executed without a human approval.

Teams build a repeatable workflow that takes a device list and target versions, runs pre-checks, clears space, stages verified firmware, upgrades HA pairs in the right order inside an approved window, verifies against a golden snapshot and ships an evidence pack, with no reboot executed without human approval.

Gennoor Tech TRAIN · INNOVATE · BUILD · For more info - https://gennoor.com

### Milestones

1. Readiness report
2. Golden snapshots and backups
3. Approved space clear and staging
4. HA-aware upgrade of at least one pair per vendor
5. Post-check verdicts and one rollback
6. Scheduled batch and evidence pull request

### Evaluation notes

- Evidence and safety outweigh code style
- Every destructive step must trace to an Approved change
- Check the ACI plan follows APIC-first and pair separation
- Limitations must name Anaya LB and anything run only on recorded responses

### Assessment weights

| Assessment area | Weight | Evidence |
|---|---|---|
| Working functionality | 25% | Demonstrable end-to-end workflow in a repeatable lab |
| Security and safety controls | 20% | Threat model, authorization boundaries, approval gates, safe failure |
| Testing and evaluation | 20% | Automated tests, golden dataset, measured quality and known limitations |
| Engineering quality | 15% | Clear interfaces, error handling, reproducible setup, maintainable code |
| Evidence and explainability | 10% | Traceable findings, sources, assumptions, logs |
| Documentation and presentation | 10% | Architecture diagram, README, runbook, demo, lessons learned |

## 7. Promotional video: research behind the script

**Opening scenario.** The two a.m. disk-full scenario is a teaching scenario, not a reported incident. It is realistic because NetScaler needs 7 GB free in /var for an upgrade [4] and its own guidance names leftover traces, cores and old firmware as common space consumers [15], while F5 images must sit in /shared/images before install [1].

GenNoor Tech
INNOVATE. BUILD.
For more info - https://gennoor.com

| Claim or on-screen text | Basis |
|---|---|
| 11 build modules; 3 days; 4 vendor families | Matches the outline: eleven modules over three days covering F5, NetScaler, Radware and Cisco ACI. |
| F5 boot volumes and UCS archives, NetScaler build files, Alteon image banks, ACI firmware groups | F5 volumes and UCS [9][11]; NetScaler builds in /var/nsinstall [14]; Alteon image selection for next boot [5]; ACI switch update groups [20]. Current ACI releases call them update groups. |
| lb-f5-01 BIG-IP 16.1.4 ... /shared: 91% used; 1 CMDB mismatch | Illustrative lab output with fictional hosts and values. |
| UCS backup saved, snapshot #117; gate READY | UCS via tmsh save sys ucs [11]; snapshot and gate are course design. |
| Delete BIGIP-15.1.8.iso, 2 old UCS, 3 core files, frees 4.2 GB | F5 supports deleting images and UCS archives [10][11]; sizes are illustrative. |
| upload 100%, sha256 match, image installed to HD1.2 (inactive) | Install to an inactive volume is F5 practice [1]. F5's upgrade guide uses md5 checksums [2]; we could not confirm F5 publishes SHA256 for BIG-IP ISOs, so the course compares against whatever algorithm the vendor publishes. Treat 'sha256' in the promo as illustrative. |
| Standby first, failover, then active | F5 and NetScaler both prescribe this order [1][3][12]. |
| HA sync CHANGES PENDING after standby upgrade; verdict WARN, sync deferred | F5 mirroring does not work until all devices run the new version [2] and NetScaler disables sync across differing HA versions [3]; the exact status string is illustrative. |
| ServiceNow change windows; runs through GitHub Actions; refuse outside window | Change start and end dates [28]; Actions schedule and workflow_dispatch [29]; refusal is course design. |
| ACI pre-upgrade validator PASS; APIC first; firmware group pod1-leafs | Validator and APIC-first order [20][21]; group name illustrative. |
| Pause at safe step when window closes; reboots without approval: 0 | Course design demonstrated in labs. |
| Upgrade every device the same safe way | Marketing call to action, not a factual claim. |

## References

**[1]** Installing BIG-IP Version 17.5.1.4. F5 (techdocs.f5.com). https://techdocs.f5.com/en-us/bigip-17-5-1-4/big-ip-release-notes/big-ip-install.html

Supports: Lifecycle; F5 staging and install. The source system must run 15.x or later; the .iso goes in /shared/images; save a UCS in /var/local/ucs and copy it off the device, upgrade the standby first and only then the active, turn off mirroring, and install with tmsh install sys software image <iso> volume HD1.x to an inactive location; watch progress with tmsh show sys software and read failures in /var/log/liveinstall.log.

**[2]** BIG-IP System: Upgrading Software (device group upgrade). F5 (techdocs.f5.com). https://techdocs.f5.com/kb/en-us/products/big-ip_ltm/manuals/product/bigip-system-upgrading-software-12-0-0/2.html
Supports: Lifecycle; HA ordering. A device is forced offline before upgrade (it terminates connections and accepts no new ones, and the state persists across reboots on 11.1.0 and later); mirroring does not work until every device in the group has rebooted onto the new version, so F5 recommends a maintenance window; preparation includes verifying the .iso with an md5 checksum and checking RAID health.

**[3]** Upgrade a NetScaler high availability pair. NetScaler (docs.netscaler.com). https://docs.netscaler.com/en-us/citrix-adc/current-release/upgrade-downgrade-citrix-adc-appliance/upgrade-downgrade-ha-pair
Supports: HA ordering. Upgrade the secondary first, confirm with show ha node that it is secondary with synchronization disabled, run force failover so it becomes primary, then upgrade the former primary; when builds have different internal HA versions, synchronization and propagation are disabled until both match.

**[4]** Before you begin (NetScaler upgrade). NetScaler (docs.netscaler.com). https://docs.netscaler.com/en-us/citrix-adc/current-release/upgrade-downgrade-citrix-adc-appliance/upgrade-downgrade-before-you-begin
Supports: Pre-flight; space clear. /var needs 7 GB free (1 GB bundle, 2 GB extraction, 4 GB upgrade) and /flash needs at least 250 MB; NetScaler recommends upgrading one major release at a time (for example 12.1 to 13.1 to 14.1) and backing up and removing /var/customizations before upgrading.

**[5]** AlteonOS 33.5.2.0 Release Notes: Upgrade Path. Radware. https://portals.radware.com/ReleaseNotes/Alteon_Release_Notes_33_5_2_0/Alteon_Release_Notes/AlteonOS-33_5_2_0-Release_Notes.1.03.html
Supports: Lifecycle; Alteon rollback. Back up the configuration and run the Upgrade Advisor Tool (updated database) before upgrading; 31.x to 33.x upgrade directly while older versions need intermediate hops; configuration rollback is not supported, so a version rollback means setting the next-boot configuration to factory, selecting the old image, rebooting and uploading the saved configuration.

**[6]** Updating Device Software (Radware device web help v6.13). Radware. https://webhelp.radware.com/lp/v613/Index/wwhelp/wwhimpl/js/Updating_Software.htm
Supports: Lifecycle (older Radware device families). This Radware device help page requires an upgrade password from Radware tied to the base MAC address and the software file size, a saved configuration before upgrading, and lets the operator choose whether to enable the new version immediately; the device then reboots and the console confirms the version.

**[7]** SP 800-40 Rev. 4: Guide to Enterprise Patch Management Planning. NIST. https://csrc.nist.gov/pubs/sp/800/40/r4/final
Supports: Why this course. Published April 2022, it defines enterprise patch management as identifying, prioritizing, acquiring, installing and verifying patches, updates and upgrades, and frames it as preventive maintenance to be operationalized with an enterprise strategy.

**[8]** APIRef_tm_sys_software_image. F5 (clouddocs.f5.com). https://clouddocs.f5.com/api/icontrol-rest/APIRef_tm_sys_software_image.html
Supports: Inventory; staging. /mgmt/tm/sys/software/image lists images (resource methods GET, DELETE, POST) with properties build, checksum, fileSize, product, verified (yes if the image is authentic), version and volume.

**[9]** APIRef_tm_sys_software_volume. F5 (clouddocs.f5.com). https://clouddocs.f5.com/api/icontrol-rest/APIRef_tm_sys_software_volume.html
Supports: Inventory; install; rollback. /mgmt/tm/sys/software/volume reports each boot location with active, version, build, basebuild, product, status (complete, installing and so on) and /media/default-boot-location.

**[10]** sys software image (tmsh reference). F5 (clouddocs.f5.com). https://clouddocs.f5.com/cli/tmsh-reference/latest/modules/sys/sys_software_image.html
Supports: Space clear; install. tmsh install sys software image takes create-volume, reboot and volume options;

list image can show checksum and verified; delete image removes named images or all; mirrored volume names must begin with MD1.

**[11]** sys ucs (tmsh reference). F5 (clouddocs.f5.com). https://clouddocs.f5.com/cli/tmsh-reference/latest/modules/sys/sys_ucs.html
Supports: Pre-checks; rollback. save ucs <name> writes the running configuration to /var/local/ucs/<name>.ucs; load ucs restores it (options include no-license and passphrase); list ucs and delete ucs manage archives in /var/local/ucs.

**[12]** sys failover (tmsh reference). F5 (clouddocs.f5.com). https://clouddocs.f5.com/cli/tmsh-reference/latest/modules/sys/sys_failover.html
Supports: HA ordering. run sys failover standby makes the active unit standby so its peer becomes active; run failover also supports offline and online.

**[13]** Demystifying iControl REST Part 5: Transferring Files. F5 DevCentral. https://community.f5.com/kb/technicalarticles/demystifying-icontrol-rest-part-5-transferring-files/286689
Supports: Staging; backups. Image files upload with POST /mgmt/cm/autodeploy/software-image-uploads/ to /shared/images/; UCS archives download with GET /mgmt/shared/file-transfer/ucs-downloads/ from /var/local/ucs/; transfers are chunked with Content-Range: start-end/filesize and anything over 1M must be chunked.

**[14]** Upgrade a NetScaler standalone appliance. NetScaler (docs.netscaler.com). https://docs.netscaler.com/en-us/citrix-adc/current-release/upgrade-downgrade-citrix-adc-appliance/upgrade-standalone-appliance
Supports: Staging; install. Run save config and copy ns.conf off the box, create /var/nsinstall/<release>/build_<build>, copy and extract the build .tgz, run ./installns --pre_check for pre-upgrade validation and then ./installns; the installer asks whether to delete old signature files and kernel images, and NetScaler advises answering No in most cases.

**[15]** How to free space on /var directory. NetScaler (docs.netscaler.com). https://docs.netscaler.com/en-us/citrix-adc/current-release/system/troubleshooting-citrix-adc/how-to-free-space-on-var-directory.html
Supports: Space clear. Check with df -h and free space when /var reaches 90%; /var/nstrace is the most common cause of a full disk; clear /var/core and /var/crash unless recent investigation needs them; clear /var/nsinstall except the firmware currently in use; delete unneeded support bundles in /var/tmp/support.

**[16]** install (NITRO API). NetScaler (developer-docs.netscaler.com). https://developer-docs.netscaler.com/en-us/adc-nitro-api/current-release/configuration/utility/install.html
Supports: Install. POST /nitro/v1/config/install takes a mandatory url for the build file (http, https, sftp, scp, ftp or file:///), y to skip the reboot prompt, dontreboot, and Async, which returns an install id to track progress with show ns job.

**[17]** systembackup (NITRO API). NetScaler (developer-docs.netscaler.com). https://developer-docs.netscaler.com/en-us/adc-nitro-api/current-release/configuration/system/systembackup.html
Supports: Pre-checks; backups. POST /nitro/v1/config/systembackup?action=create makes a .tgz backup with filename, level (basic or full, default basic), includekernel and comment.

**[18]** hanode (NITRO API). NetScaler (developer-docs.netscaler.com). https://developer-docs.netscaler.com/en-us/adc-nitro-api/current-release/configuration/ha/hanode.html
Supports: Inventory; HA ordering. hanode exposes the read-only master state (Primary, Secondary, UNKNOWN), hastatus (ENABLED, STAYSECONDARY, STAYPRIMARY, DISABLED), hasync and the reason for any HA sync failure.

**[19]** reboot (NITRO API). NetScaler (developer-docs.netscaler.com). https://developer-docs.netscaler.com/en-us/adc-nitro-api/current-release/configuration/ns/reboot.html
Supports: Install. POST /nitro/v1/config/reboot reboots the appliance; warm: true restarts the NetScaler software without rebooting the operating system.

**[20]** Cisco APIC Installation and ACI Upgrade and Downgrade Guide: Pre-upgrade checklists. Cisco. https://www.cisco.com/c/en/us/td/docs/dcn/aci/apic/all/apic-installation-aci-upgrade-downgrade/Cisco-APIC-Installation-ACI-Upgrade-Downgrade-Guide/m-pre-upgrade-checklists.html

Supports: Cisco ACI. The APIC pre-upgrade validator runs automatically when an update group is configured and a standalone script can be run on the APIC; export the configuration with AES encryption first; APICs are upgraded before switches; since 4.0(1) there is a single switch update group type; upgrade one switch of each HA pair at a time; all switches must be Active under Fabric Membership.

**[21]** ACI Pre-Upgrade Validation Script. Cisco datacenter (GitHub Pages). https://datacenter.github.io/ACI-Pre-Upgrade-Validation-Script/
Supports: Cisco ACI. Copy aci-preupgrade-validation-script.py to the APIC (suggested /data/techsupport), run it with python, give admin credentials and the target version (which must already be on the APIC), and act on every FAIL or MANUAL CHECK REQUIRED result; output totals PASS, FAIL - OUTAGE WARNING, FAIL - UPGRADE FAILURE, MANUAL CHECK REQUIRED, N/A and ERROR.

**[22]** ACI Pre-Upgrade Validation Script: Validation reference. Cisco datacenter (GitHub Pages). https://datacenter.github.io/ACI-Pre-Upgrade-Validation-Script/validations/
Supports: Cisco ACI. Checks include target image and MD5, APIC cluster Fully Fit, switch fabric membership, NTP, switch upgrade group guidelines, APIC and switch disk and /bootflash usage; the built-in validator runs during APIC or switch upgrades, and the Pre-Upgrade Validator app was deprecated in ACI 6.1(2); APICs are upgraded first prior to switches.

**[23]** ACI Sandboxes. Cisco DevNet. https://developer.cisco.com/docs/aci/sandbox
Supports: Lab environment. The Always-On ACI Simulator sandbox accepts APIC API requests at any time with no reservation, as a shared instance.

**[24]** ACI Programmability: Getting Started. Cisco DevNet. https://developer.cisco.com/docs/aci/getting-started/
Supports: Cisco ACI; MCP tools. APIC REST authentication is POST /api/aaaLogin.json with an aaaUser object holding name and pwd attributes.

**[25]** Table API. ServiceNow. https://www.servicenow.com/docs/r/api-reference/rest-apis/c_TableAPI.html
Supports: Inventory; guardrails; scheduler. The Table API provides GET, POST, PUT, PATCH and DELETE on /api/now/table/{tableName}; GET filters with sysparm_query (for example field=value^ORDERBYfield), trims with sysparm_fields and pages with sysparm_limit and sysparm_offset; POST inserts one record and returns 201.

**[26]** Load balancer fields and probes (CMDB tables). ServiceNow. https://www.servicenow.com/docs/r/guxhDnVk8OUAJKLm7WZsYA/Gaji~uTPELoKJAs~J7YyCA
Supports: Inventory. Load balancer CIs are stored in cmdb_ci_lb with classes such as cmdb_ci_lb_bigip (F5 BIG-IP) and cmdb_ci_lb_netscaler (Citrix NetScaler), plus related tables cmdb_ci_lb_pool, cmdb_ci_lb_pool_member, cmdb_ci_lb_service, cmdb_ci_lb_interface, cmdb_ci_lb_vlan and cmdb_ci_lb_appl.

**[27]** Ask for Approval action. ServiceNow. https://www.servicenow.com/docs/r/build-workflows/workflow-studio/ask-approval-flow-designer.html
Supports: Guardrails. Ask for Approval takes a record, table, approval and rejection rules and an optional due date, and outputs an approval state (Requested, Approved, Rejected, Cancelled, No Longer Required, Skipped); running several Ask for Approval actions on the same record in parallel branches is discouraged.

**[28]** Change Management API. ServiceNow. https://www.servicenow.com/docs/r/api-reference/rest-apis/change-management-api.html
Supports: Guardrails; scheduler. The sn_chg_rest Change Management API creates change requests (one per call) and runs conflict checks (GET /api/sn_chg_rest/change/{sys_id}/conflict); change records carry start_date and end_date for the planned implementation window, and change model states include New, Scheduled, Implement, Review and Closed.

**[29]** Events that trigger workflows. GitHub Docs. https://docs.github.com/en/actions/reference/workflows-and-actions/events-that-trigger-workflows
Supports: Scheduler. schedule runs on the latest commit of the default branch in UTC (optional IANA timezone), no more often than every 5 minutes, and may be delayed or dropped at high load such as the top of the hour; workflow_dispatch accepts typed inputs and needs the workflow file on the default branch.

**[30]** APScheduler 3.x User guide. APScheduler project. https://apscheduler.readthedocs.io/en/3.x/userguide.html

Supports: Scheduler. APScheduler has triggers, job stores, executors and schedulers; persistent job stores survive restarts, SQLAlchemyJobStore on PostgreSQL is recommended when free to choose, and misfire_grace_time decides whether a missed run still executes.

**[31]** Pydantic: Models. Pydantic. https://docs.pydantic.dev/latest/concepts/models/
Supports: Inventory; tool outputs. Pydantic models validate data on creation or through model_validate and model_validate_json, raising ValidationError when data cannot be parsed into the model.

**[32]** Claude Code overview. Anthropic. https://code.claude.com/docs/en/overview
Supports: Workbench setup; pipeline. Claude Code is an agentic coding tool for the terminal, IDE, desktop and browser; on macOS, Linux and WSL it installs with curl -fsSL https://claude.ai/install.sh | bash, and claude -p runs it non-interactively for scripts and CI.

**[33]** Give Claude custom tools (Agent SDK). Anthropic. https://code.claude.com/docs/en/agent-sdk/custom-tools
Supports: Workbench setup. Python tools are defined with @tool, wrapped with create_sdk_mcp_server and passed in mcp_servers; their names are mcp__{server_name}__{tool_name} and are pre-approved through allowed_tools; tools can set readOnlyHint, return structuredContent and return isError: true on failure.

**[34]** MCP schema reference: ToolAnnotations. Model Context Protocol. https://modelcontextprotocol.io/specification/2025-06-18/schema
Supports: Workbench setup; guardrails. ToolAnnotations provide readOnlyHint, destructiveHint, idempotentHint and openWorldHint; all are hints, not guarantees, and clients should never make tool-use decisions based on annotations from untrusted servers.

**[35]** Structured outputs. Anthropic. https://platform.claude.com/docs/en/build-with-claude/structured-outputs
Supports: Upgrade plan; post-check verdict. JSON outputs send a JSON schema in output_config.format (type json_schema) so the response matches the schema, and strict tool use (strict: true) guarantees schema validation of tool names and inputs.

**[36]** Hooks reference. Anthropic. https://code.claude.com/docs/en/hooks
Supports: Guardrails. PreToolUse hooks run after Claude creates tool parameters and before the call is processed, match built-in and MCP tool names, and can block the call (exit code 2) or return permissionDecision deny with a reason.

**[37]** LLM06:2025 Excessive Agency. OWASP Gen AI Security Project. https://genai.owasp.org/llmrisk/llm062025-excessive-agency/
Supports: Guardrails. Mitigations include minimizing extension functionality and permissions and requiring human-in-the-loop approval of high-impact actions, implemented in a downstream system or the extension itself.

**[38]** alteon-sdk (Python package metadata). PyPI (package copyright Radware Ltd). https://pypi.org/pypi/alteon-sdk/json
Supports: MCP tools; Alteon adapter. alteon-sdk wraps the Alteon REST backend with configurators, management functions for facts collection and operations (for example client.api.mgmt.info.software and config commit) and deploy options dry_run, write_on_change and get_diff; minimum supported Alteon versions are 31.0.10.0 and 32.2.2.0.

**[39]** cm config-sync (tmsh reference). F5 (clouddocs.f5.com). https://clouddocs.f5.com/cli/tmsh-reference/latest/modules/cm/cm_config-sync.html
Supports: HA ordering; post-checks. run cm config-sync starts a configuration synchronization job and must be given one of to-group, from-group or recover-sync to set the direction.

**[40]** NetScaler licensing (VPX Express). NetScaler (docs.netscaler.com). https://docs.netscaler.com/en-us/citrix-adc/current-release/licensing
Supports: Lab environment. VPX Express needs no license file and offers 20 Mbps bandwidth, a maximum of 250 SSL sessions and 20 Mbps SSL throughput; from release 14.1 build 29.65 it includes all Premium-license features.

**[41]** Understanding PDIs. ServiceNow. https://www.servicenow.com/docs/r/SmYvGPN0OygSQs53fsexIQ/yG_RdLKb649OcfnDiGEkwQ
Supports: Lab environment. Personal Developer Instances are free for Developer Program members; an instance

unused for ten days is reclaimed and reset, and idle instances hibernate until the owner signs in to the Developer Site.

**[42]** How Claude remembers your project (CLAUDE.md). Anthropic. https://code.claude.com/docs/en/memory Supports: Workbench setup. A project CLAUDE.md lives at ./CLAUDE.md or ./.claude/CLAUDE.md, is shared through source control and should hold build and test commands, coding standards, naming conventions and common workflows.

For more info - https://gennoor.com

Gennoor Tech
TRAIN · INNOVATE · BUILD
For more info - https://gennoor.com